



Task: Hibernate "ddl-auto"

أولاً: يعني إيه ddl-auto؟

الخاصية دي بتحدد **Hibernate** هيتعامل إزاي مع الـ Database Schema بناءً على الـ Entities الموجودة في الـ Java.

مثال:

```

@Entity
public class Employee {

    @Id
    private Long id;

    private String name;

    private double salary;
}

```

Hibernate محتاج يعرف:

هل ينشئ جدول؟ هل يعدّل الجدول الموجود؟ هل يتأكد فقط إن الجدول مطابق؟ أم لا يفعل شيئاً؟
وده تحديدًا دور:

spring.jpa.hibernate.ddl-auto=...

Validate(spring.jpa.hibernate.ddl-auto=validate)

بتعمل إيه؟

Hibernate لا ينشئ ولا يعدّل ولا يحذف أي شيء في الـ Database.

هو فقط يتأكد أن الـ Database Schema الحالية متوافقة مع الـ Entities.

```

@Entity
public class Employee {

    @Id
    private Long id;

    private String name;
}

```

إزاي بتشتغل؟

مثلاً عندك: Entity

والـ Database فيها:

Hibernate يقول

تمام، الـ Schema متوافقة.

employee

id

name

```
private String email;
```

لكن لو ال Entity فيها: -----
وال Database مفيهاش email:

Hibernate يكتشف المشكلة ويوقف تشغيل التطبيق بسبب عدم تطابق ال Schema.
:- Real-world Case

مناسب جدًا في Production لما تكون ال Database مُدارة باستخدام أدوات migrations مثل:

• Flyway

• Liquibase

يعني ال Database يتم تعديلها بطريقة منظمة، و Hibernate دوره فقط يتأكد أن ال Entities متوافقة معها.

Update (spring.jpa.hibernate.ddl-auto=update)

بتعمل إيه؟

Hibernate يحاول تعديل ال Database Schema الموجودة لتتوافق مع ال Entities.

إزاي بتشتغل؟

```
@Entity
public class Employee {

    @Id
    private Long id;

    private String name;
}
```

employee

id

مثلاً عندك: Entity

وال Database فيها: -----
Hibernate ممكن يكتشف أن name غير موجود ويقوم بإضافته.

```
ALTER TABLE employee ADD name VARCHAR(...);
```

كل مرة يبدأ فيها:

1. **Hibernate** يقرأ الـ Entities.

2. يفحص الـ Schema الموجودة .

3. يقارن بينهم .

4. يحاول إجراء التعديلات اللازمة.

```
private String phone;
```

لو أضفت:

Hibernate قد يقوم بإضافة: column الى

phone

الجدول

- :Real-world Case

يمكن استخدامه في:

• Development

• Local environment

• Prototype

• مشروع صغير أثناء التطوير

لكن لا يُفضّل الاعتماد عليه في Production في المشاريع الجادة.

لأن **Hibernate** هو الذي يحاول تحديد وتنفيذ تغييرات الـ Schema ، وهذا أقل أماناً وتحكماً من استخدام migrations.

Create (spring.jpa.hibernate.ddl-auto=create)

بتعمل إيه؟

Hibernate يقوم بإنشاء الـ Database Schema من جديد عند بدء التطبيق.

يعني الجداول الموجودة التي يديرها Hibernate يتم حذفها ثم إنشاؤها.

إزاي بتشتغل؟

```
@Entity
public class Employee {

    @Id
    private Long id;

    private String name;
}
```

مثلاً عندك: Entity

Hibernate ينشئ جدولاً مشابهاً لـ:

```
CREATE TABLE employee (
    id BIGINT PRIMARY KEY,
    name VARCHAR(...)
);
```

الخطر المهم جداً

لو كان عندك Data موجودة:

واستخدمت create، فالـ Schema ستُعاد تهيئتها، وبالتالي البيانات الموجودة قد تُفقد.

- Real-world Case

ممكن تستخدمه في:

• Development , Testing

• Demo environments أو مشروع جديد لا يهم فيه الاحتفاظ بالبيانات

لكن لا تستخدمه على Production Database تحتوي على بيانات مهمة.

Create-drop (spring.jpa.hibernate.ddl-auto= create-drop) و

بتعمل إيه؟

دي مشابهة جدًا لـ create لكن فيها خطوة إضافية.

```
DROP existing schema
↓
CREATE new schema
```

عند Startup:

```
DROP schema
```

وعند Shutdown:

يعني الـ Schema يتم إنشاؤها عند بداية التطبيق ثم حذفها عند إيقاف التطبيق.

- Real-world Case :

ممتاز في Automated Tests.

مثلاً: Integration Test

```
Start Test
↓
Create Database Schema
↓
Run Tests
↓
Shutdown
↓
Delete Schema
```

وبالتالي كل Test Environment يبدأ من حالة نظيفة.

None (spring.jpa.hibernate.ddl-auto= none)

بتعمل إيه؟

Hibernate لا يفعل أي شيء متعلق بالـ Database Schema.

لا --CREATE, ALTER, DROP, VALIDATE

Hibernate يترك إدارة الـ Schema بالكامل لأداة أخرى أو للـ Database نفسها.

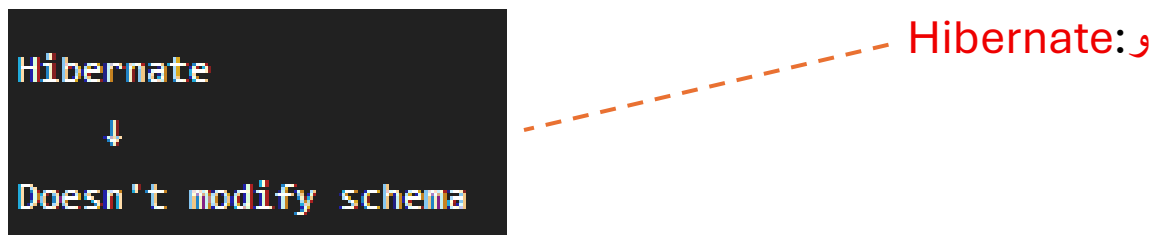
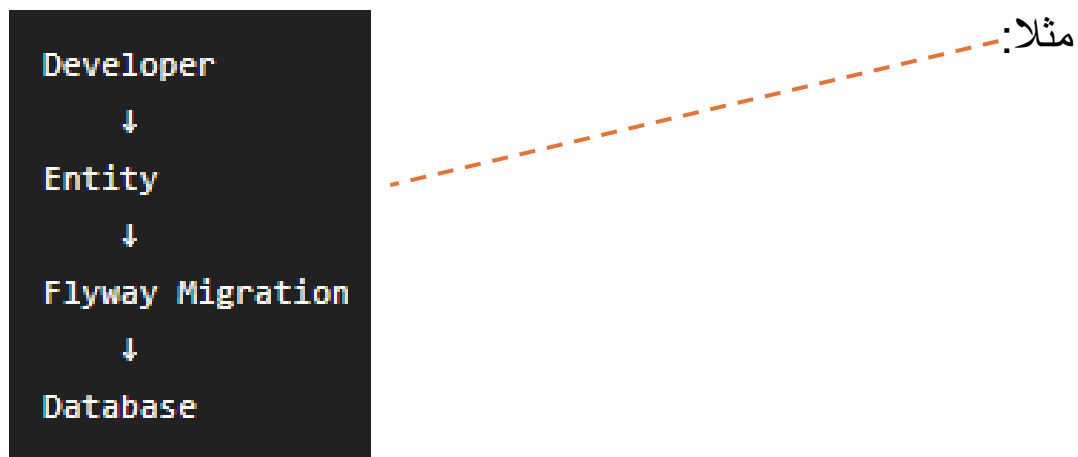
- Real-world Case :

Real-world Case

مناسب جدًا عندما تستخدم:

- Flyway
- Liquibase

لإدارة Database migrations.



★ المثال المطلوب في التاسك:

لو عندك: `spring.jpa.hibernate.ddl-auto=update`

Hibernate عند تشغيل التطبيق سيحاول جعل الـ Database Schema متوافقة مع الـ Entities.

مثلا

```
product
-----
id
name
```

ولو الـ Database فيها:

price

فقد يضيف: **Hibernate**

```
@Entity
public class Product {

    @Id
    private Long id;

    private String name;

    private double price;
}
```

بحيث تصبح:

```
product
-----
id
name
price
```

Real-world case

تخيل أنك تعمل على مشروع أثناء مرحلة **Development**.

```
private String name;
```

اليوم عندك:

```
private String description;
```

وبعد يوم قررت تضيف

بدل ما تعدل الـ Database يدويًا كل مرة، `update` يمكنه محاولة إضافة الـ column الجديد.

لذلك هو مريح جدًا أثناء التطوير.

Real-world Case	بتعمل إيه؟	Value
Production مع Flyway/Liquibase	تتأكد أن الـ Schema متوافق مع الـ Entities بدون تعديل	validate
Development / Prototype	تعدل الـ Schema الموجودة لتناسب الـ Entities	update
Development / Demo / Testing	ت حذف الـ Schema وتعيد إنشاءها عند Startup	create
Automated Integration Tests	تنشئ الـ Schema عند Startup وتحذفها عند Shutdown	create-drop
Production مع Flyway/Liquibase	Hibernate لا يدير الـ Schema نهائياً	none

validate	→ فقط Check
update	→ Modify
create	→ Recreate
create-drop	→ Create ثم Drop
none	→ Do Nothing